```
        callback params...
      fn paramsCopy...
```

Now you can define a cached `loadProductData` by using `withCachedCallback`:

```
loadProductData = withCachedCallback ((makeDbOperator products)
'get')
```

This function will now cache each response forever. If you want it to cache items for a fixed time, you need to store each cache item with an expiry time:

```
withExpiringCachedCallback = (fn, ttl) ->
  cache = Object.create null
  (params...) ->
    key = params[0]
    callback = params[params.length - 1]
    if cache[key]?.expires > Date.now()
      callback cache[key].entry...
    else
      paramsCopy = params[..]
      paramsCopy[params.length - 1] = (params...) ->
        console.log params
        cache[key] =
          entry: params
          expires: Date.now() + ttl
        console.log cache[key]
        callback params...
      fn paramsCopy...
```

This caching technique can be applied in a general sense to any function, where it is known as *memoization*.

Memoization

Caching the evaluation of a function with specific arguments is called memoization. You've seen it used specifically to cache the loading of some data. To reinforce the concept, it's worth also seeing a more abstract numerical example. Though not practical or exciting, memoizing a factorial function is a good way to understand this: